

Kassidy Shomo

Professor Kraya

CS 499: Computer Science Capstone

June 18, 2026

Enhancement One: Software Design/Engineering

This artifact was originally developed on January 15 as part of CS 405: Secure Coding. At the time, the program was intended as a demonstration of a buffer overflow vulnerability in C++ using a fixed-size character array. The original implementation showed how unsafe user input could exceed a defined buffer size and overwrite adjacent memory. This included the potential to corrupt other variables in the program, even when those variables were intended to remain constant. The purpose of the original version was to highlight how quickly memory corruption can occur in low-level programming when input validation is not properly handled.

The reason I selected this artifact for my ePortfolio is because it clearly demonstrates both a common security vulnerability and an opportunity for improvement through software design. Buffer overflows are a real-world issue in systems programming and security-focused development, and this program provides a direct example of how they occur and how they can be prevented. The artifact also demonstrates improvement from a vulnerable implementation to a more structured and secure version that follows stronger software engineering practices.

In the earlier version of the artifact, the primary issue was that user input could be copied into a fixed-size buffer without enough protection against unsafe memory access or structured

validation. This enhancement builds on that foundation by expanding validation logic and improving control flow for input handling. The updated implementation directly addresses unsafe memory handling by validating input before any copy operation occurs. This is implemented through the helper function `safe_copy_to_buffer`, which checks the input length against `USER_BUFFER_SIZE` before performing the bounded copy operation. As shown in the enhanced code, this validation occurs here:

```
if (input.length() >= buffer_size) { return false; }
```

This helps prevent unsafe input from reaching the fixed buffer char `user_input[USER_BUFFER_SIZE]`, which was the original vulnerability point in the program. The enhanced version also checks for empty input before processing continues, which improves consistency and prevents invalid program states from being accepted.

Another major improvement is the separation between validation logic and the main program flow. The helper function focuses specifically on validating and safely copying data into memory, while main manages user interaction, invalid attempt tracking, and overall control flow. This improves readability and maintainability.

The enhanced program also improves how invalid input is handled. Instead of simply rejecting unsafe data and continuing execution without structure, the program now tracks repeated invalid attempts through an `attempt_count` variable. Empty input and overflow attempts both increase this count, while successful validated input resets it. If the maximum number of invalid attempts is reached, the program safely terminates execution with the message:

```
"Too many invalid attempts. Program terminating."
```

This creates more controlled program behavior and reflects how real systems often enforce restrictions after repeated invalid activity.

When an overflow attempt is detected, the program explicitly notifies the user with:

"Buffer overflow attempt detected and blocked."

The program also preserves safe behavior by maintaining a `last_safe_value` variable. This ensures that when invalid input is rejected, the program can still display the most recent valid state instead of corrupted or unsafe data. This replaces the risk present in the original version where invalid input could overwrite memory and produce unpredictable results.

The core improvements can be summarized through the following changes between the original and enhanced code:

Original	Enhanced
direct unsafe copy into <code>char user_input[USER_BUFFER_SIZE]</code>	validated copy using <code>safe_copy_to_buffer</code> before writing to memory
limited validation structure	improved validation flow with empty input and overflow handling
basic overflow prevention	controlled invalid attempt tracking and termination logic
minimal control over invalid states	enforced program termination after repeated invalid input
vulnerable memory handling	controlled memory access with pre-check validation

These changes demonstrate a clear shift from a vulnerability-focused example to a more complete secure design implementation.

This artifact aligns strongly with software engineering and design because it focuses on improving program structure, memory safety, and input handling. The design changes improve reliability by ensuring unsafe input is blocked before it can affect program state. They also improve maintainability by separating responsibilities into clearer functional areas instead of handling all behavior inside a single loop. The enhancement also reflects security-focused design decisions that prioritize predictable program behavior under unsafe conditions.

This enhancement aligns with Course Outcomes 3, 4, and 5. It demonstrates Outcome 3 by applying structured logic to solve a defined problem involving unsafe input handling while considering trade-offs between usability and strict validation. It aligns with Outcome 4 through the implementation of secure coding techniques and improved software design practices. It aligns with Outcome 5 by demonstrating a security mindset, specifically by identifying a memory safety vulnerability and implementing preventative safeguards rather than reactive fixes.

Through this process, I learned how important early validation is in preventing unsafe memory operations. One of the main challenges was ensuring that validation occurs before any data reaches the fixed buffer while still keeping the program readable and maintainable. Another challenge was designing control flow that balances validation, error handling, and user interaction without making the program unnecessarily complex. Adding invalid attempt tracking also required balancing security-focused restrictions with usability. Overall, this reinforced the importance of designing software with security in mind from the beginning rather than treating it as an afterthought.